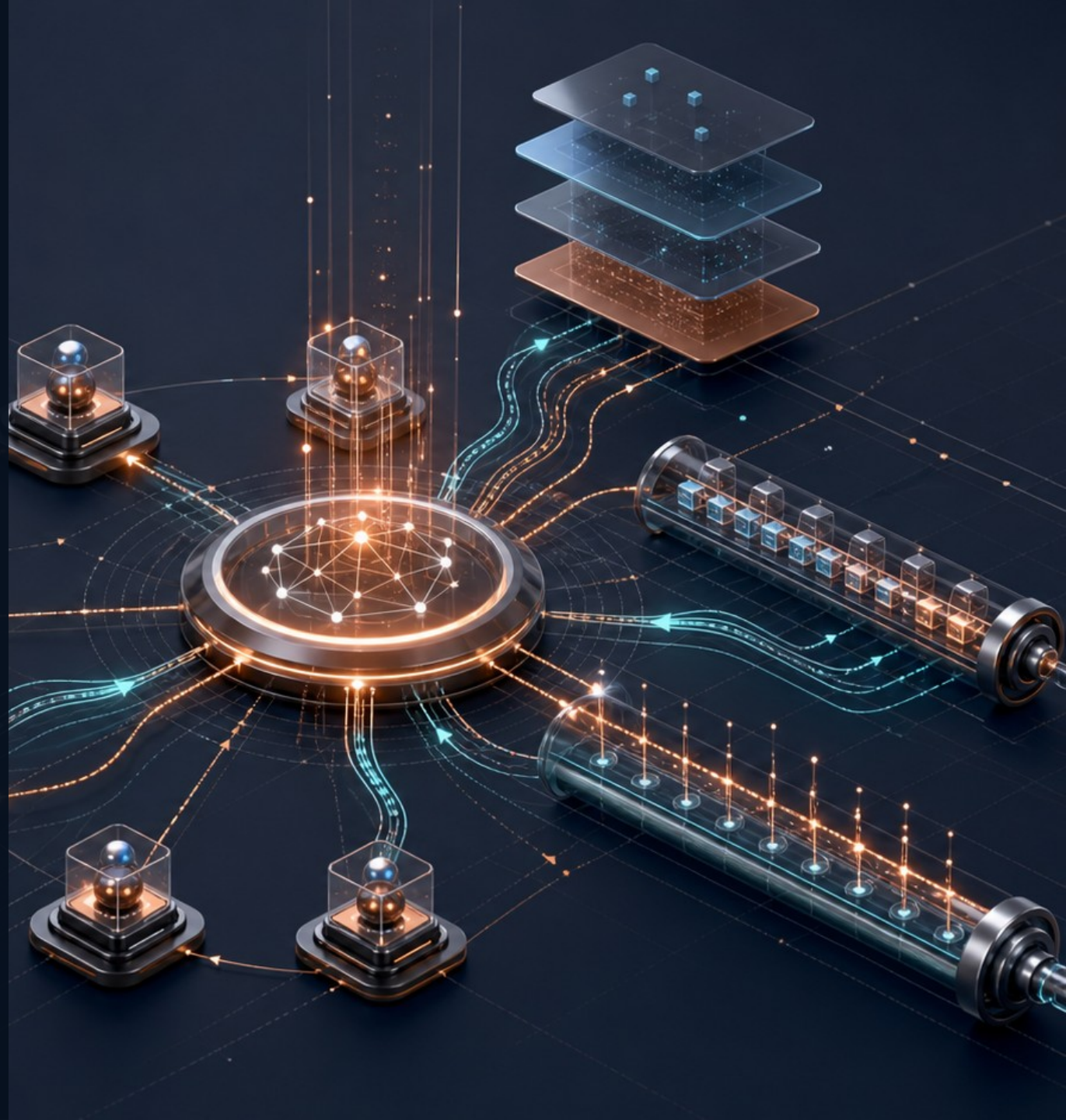


基于 **AI Agent** 的 高性能分布式通讯系统

把多 **Agent** 系统从“演示脚本”推到“可观测、可回放、可审计的日用平台”。

答辩人：以琳

日期：2026 年 5 月 9 日



问题：多 Agent 真正难在通讯底座

角色协作本身已经被证明有效，工程化短板出在状态、通讯和可观测性。

通讯协议不统一

Python 对象互调难跨进程、跨节点
A2A 提供语义，但要和本地事件系统结合

任务生命周期缺位

Agent 执行长、会中断、会等待人
需要显式 Task 状态与人工介入语义

运行时状态难追踪

工具调用、子任务、异常必须可回放
事件流必须成为一等公民

本文的问题不是“让 Agent 会聊天”，而是让多个 Agent 在本机与跨节点环境中可靠协作。

研究目标与设计假设

Rust、本地优先、事件驱动、可跨节点扩展。

在 **Rust** 工作区中实现一个本地优先、事件驱动、可观测、可跨节点扩展的多 **AI Agent** 协作平台。

13

crate 模块边界

8.25 万

约 Rust 代码行

1363 + 27

单元测试 + 集成测试

ERP

真实项目日常验证

5

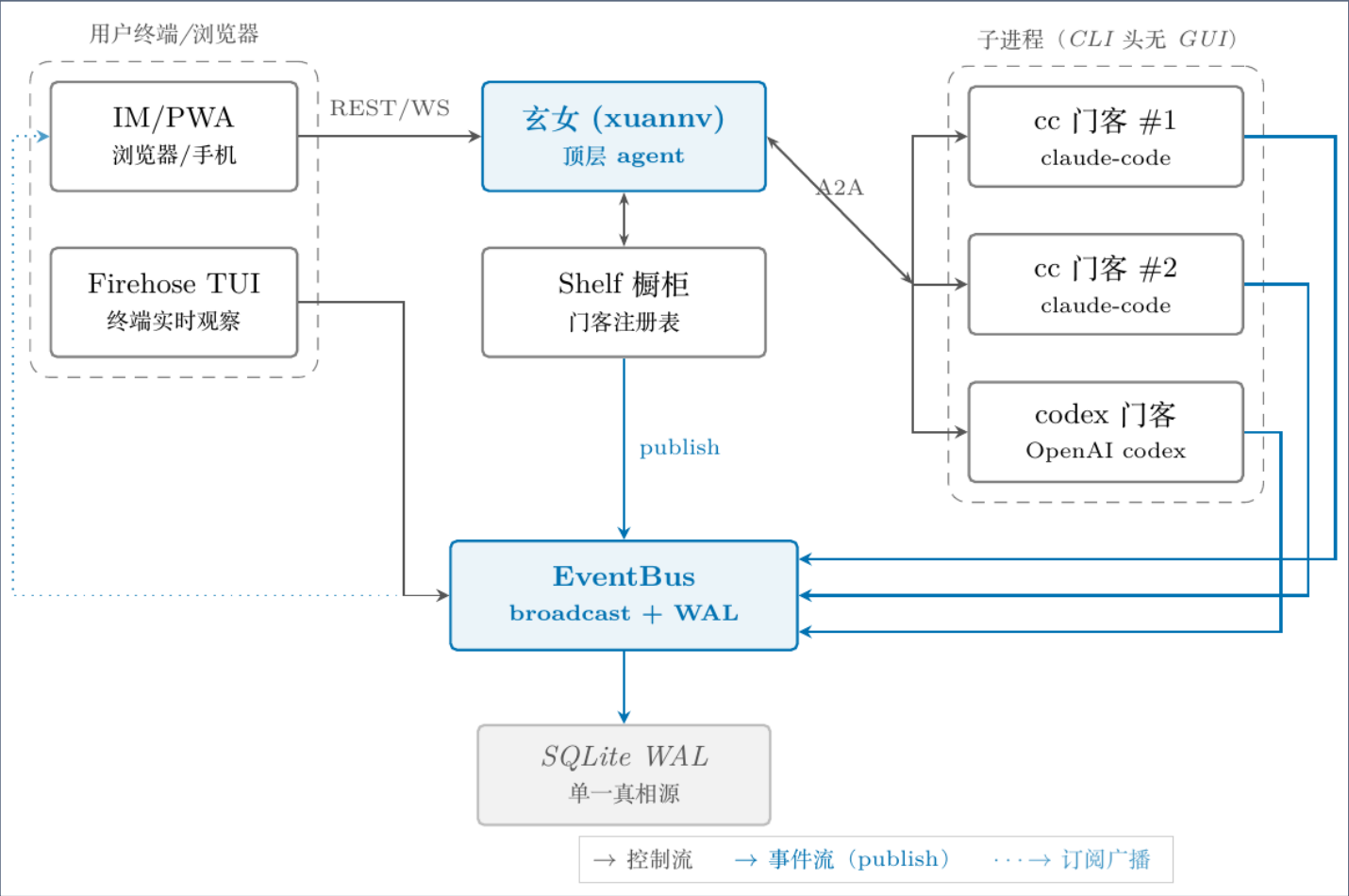
可独立审视的工程贡献

设计假设

- 事件驱动 + 追加式日志，可以同时兼顾吞吐、实时性与可追溯性。
- 协议语义、事实流、调度边界、执行隔离必须同时成立，少一个就无法日用。

总体架构：核心—通讯—编排—执行—观测

A2A 是契约语义， EventBus 是事实流；两者正交。



讲图顺序

- 底层： fuxi-core 定义 Agent / Task / Workspace / Event。
- 通讯： fuxi-events 记录事实， fuxi-a2a 承担 Agent 间契约。
- 编排： 玄女维护门客注册表，所有协作经编排层显式记录。
- 执行： Claude Code / Codex 门客适配 + git worktree 沙箱。
- 观测： Firehose 、 WebSocket 、 SSE 、 IM 共看同一事件流。

核心观点：协议不保存事实，事件流不替代协议

五项贡献：围绕一个命题展开

把 Agent 间通讯的契约、事实流、调度边界与执行隔离同时做对。

01 ————— A2A 风格协议适配层

五条核心路径；input-required 提升为平台级人工介入

02 ————— 非阻塞事件总线

Tokio broadcast + SQLite WAL ； try_send + 后台转交 + lag 哨兵

03 ————— 玄女一门客分层

用户入口与执行 Agent 解耦；编排层不可绕过

04 ————— 三层沙箱 × WS 反连

L1/L2/L3 worktree 隔离；编排层可主动控制门客

05 ————— 跨节点扩展钩子

DistEnqueuer / NodeLoadProvider 等 trait 注入；HMAC 验签

每一项贡献都能被单独评审，但它们最终服务于同一个通讯系统闭环。

A2A 适配：协议语义进入平台状态机

`input-required` 不只是 wire 字段，而是可观测、可调度的人工介入状态。

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
#[serde(rename_all = "kebab-case")]
pub enum TaskState {
    Submitted,          // 刚入队
    Working,             // 处理中
    InputRequired,       // 需要人工输入（早期 draft 未列、fuxi 提前实现）
    Completed, Failed, Canceled, // 三个终态
}
```

```
Input:  byte_stream
Output: 流式产出 ServerSentEventPayload
1: buf <- ""
2: while chunk <- byte_stream.next() do
3:     append chunk (utf8) to buf
4:     while buf 含 "\n\n" 边界 do
5:         (frame, rest) <- buf.split_at("\n\n")
6:         buf <- rest
7:         payload <- decode_frame(frame) // 多行 data: 拼接, : 注释跳过
8:         yield payload
9:     end while
10: end while
```

实现闭环

- 覆盖 agent discovery、task send、task stream、task query、task cancel 五条主路径。
- 沿用早期 A2A JSON-RPC binding；论文中明确说明与当前官方 v1.0 尚未完全对齐。
- 将 TASK_STATE_INPUT_REQUIRED 映射到 Task::PendingApproval 与 ShelfStatus::AwaitingInput。
- 采用单端点 HTTP + SSE，降低客户端连接管理复杂度。

1321 行

fuxi-a2a crate：核心 1063 + 测试 258

5 路径

发现、发送、流式订阅、查询、取消

事件总线：实时推送与历史回放共用一套抽象

publish 路径不阻塞调用方，同时让背压变成可见告警。

```
pub fn publish(&self, ev: Event) -> Result<()> {  
    // 路径 1: broadcast 零拷贝扇出，不影响 mpsc  
    let _ = self.inner.broadcast_tx.send(ev.clone());  
    // 路径 2: try_send 入 mpsc (非阻塞)  
    match self.inner.writer_tx.try_send(ev) {  
        Ok(_) => { self.inner.writer_pending.fetch_add(1, Ordering::Relaxed); }  
        Err(TrySendError::Full(dropped)) => {  
            // 拥塞：后台 spawn 等慢车位，当前 publish 立即返回  
            let pending = self.inner.writer_pending.load(Ordering::Relaxed);  
            if pending >= self.inner.cfg.lag_threshold {  
                self.maybe_emit_lag_sentinel(pending); // 哨兵告警  
            }  
            let tx = self.inner.writer_tx.clone();  
            tokio::spawn(async move { let _ = tx.send(dropped).await; });  
        }  
        Err(TrySendError::Closed(_)) => return Err(Error::WriterClosed),  
    }  
    Ok()  
}
```

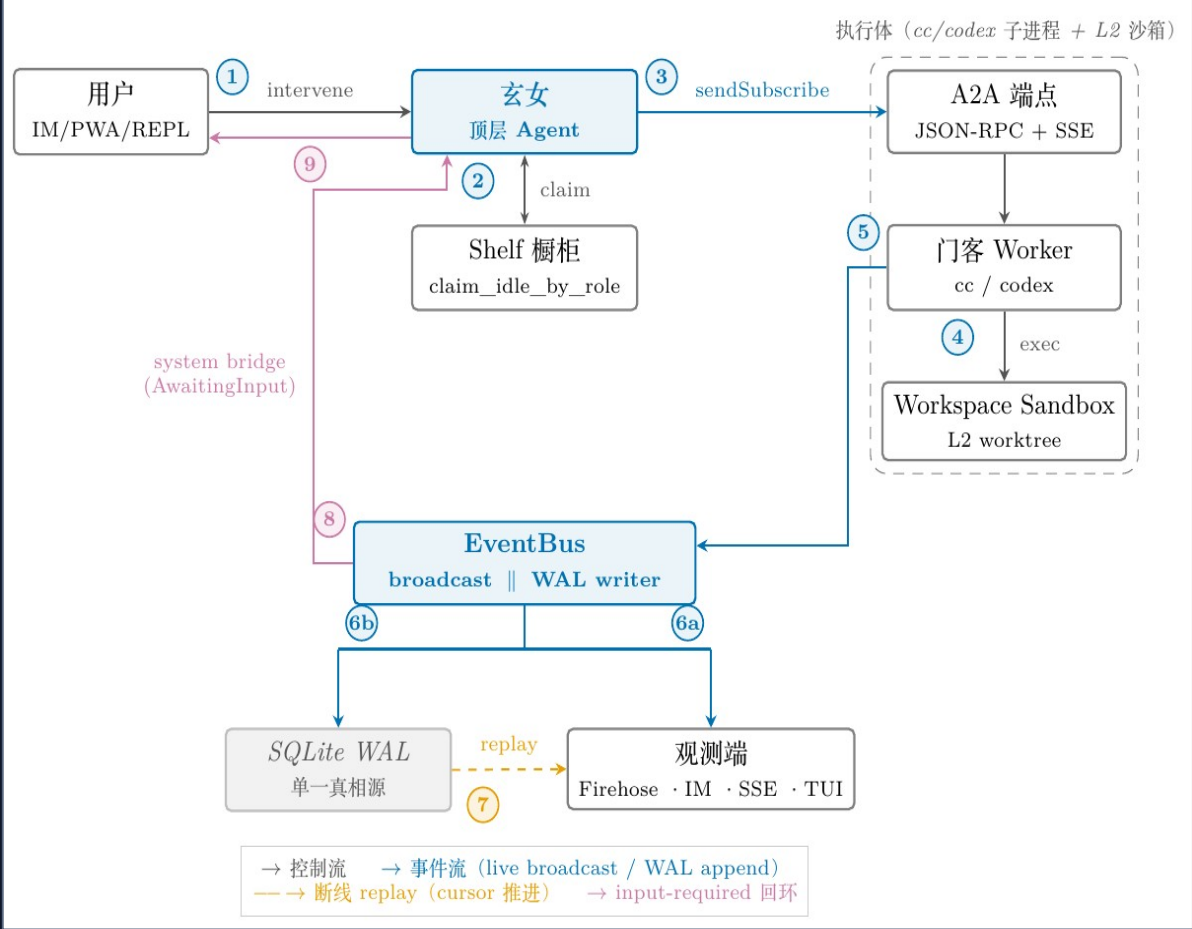
三步组合

- 1 broadcast** 零拷贝扇出给实时订阅者
- 2 try_send** 事件投递给异步 writer 落 SQLite WAL
- 3 lag sentinel** 队列超过阈值 512 时发出可观测告警

关键取舍：宁可暴露 **lag**，也不让调用方在 **publish()** 上被持久化路径拖住。

玄女—门客：编排层不可绕过

顶层 Agent 负责对话与调度，工作 Agent 负责执行；所有协作都留下事件。



工程边界

- 玄女是用户对话面唯一入口；门客之间不直接绕过编排层。
- publish-then-pump：门客私有事件补齐 task_id / agent_id 后再进入全局 EventBus。
- claim_idle_by_role 原子化避免并发派活撞同一门客。
- shutdown_agent 对玄女做硬豁免，避免 idle GC 误杀顶层调度 Agent。

```
pub async fn claim_idle_by_role(&self, role: &str) -> Option<AgentId> {  
    let mut guard = self.inner.write().await;  
    let pick = guard.iter_mut().  
        .find(|(_, e)| e.status == ShelfStatus::Idle && e.card.profile.role == role);  
    if let Some((id, e)) = pick {  
        e.status = ShelfStatus::Busy;  
        e.idle_since = None; // 占用必须清，否则 GC 误杀  
        Some(*id)  
    } else { None }  
}
```

答辩句：编排层既不是聊天路由，也不是业务逻辑，而是事实流的边界。

执行隔离：三层沙箱 × WebSocket 反连

把 headless Agent CLI 接进平台，而不是把平台绑死在某一个 CLI 上。

```
pub async fn launch_with_id(id: AgentId, profile: AgentProfile, cfg: CcLaunchConfig) -> Result<Self> {
    let channel = Arc::new(WsChannel::bind(&Uuid::new_v4().to_string()).await?);
    let mut child = spawn_claude(&cfg?); // 注入 --sdk-url + NO_PROXY (避开 Clash TUN)
    tokio::select! {
        Ok(_) = channel.wait_connect(CONNECT_TIMEOUT) => {},
        Ok(_) = child.wait() => return Err(CcError::EarlyExit.into()),
    }
    let pump = tokio::spawn(pump_loop(channel.clone(), /* ... */));
    Ok(CcAgent { card: AgentCard { id, profile, endpoint: channel.url(), .. },
        inner: Arc::new(Mutex::new(Inner { channel, child, .. })), _pump: pump })
}
```

```
/// L2 ephemeral worktree (Decision 21 phase 2) — per-task 一次性工作区。
/// 任务完后封存 (archive 24h)，再后被 GC 清理。
/// L3 PersistentSandboxManager (L3 sandbox) 的差别：
/// 维度 / L3 持久 sandbox / L2 ephemeral worktree /
/// -----|-----|-----|
/// 索引键 / (project, role) / (project, task_id) /
/// 路径 / <root>/<proj>/sandboxes / <root>/<proj>/ephemeral/<task> /
/// 分支 / <role>/<proj>-main (长期) / task/<task-uuid> (一次性) /
/// 生命周期 / 长期，跨 task 复用 / 任务完 + archive + 24h 后 GC /
/// fork 起点 / canonical <default_branch> / canonical <default_branch> /
///
/// 本模块只管文件系统 + git 操作：事件发布
/// (WorkspaceCreated/layer=L2) / WorkspaceArchived / WorkspaceCollected)
/// 由 caller 在调用前后包装——保持 fuzi-workspace 不依赖 fuzi-events。

pub struct EphemeralWorkspaceHandle {
    pub project: ProjectId,
    pub task: TaskId,
    pub canonical_path: PathBuf,
    /// <root>/<project>/ephemeral/<task-display>/ 绝对路径。
    pub workspace_path: PathBuf,
    /// task/<task-uuid> —— 一次性 branch。
    pub branch: String,
    /// 跨事件关联键：<project>/L2/<task-uuid>。
    pub workspace_id: WorkspaceId,
}
```

L1

只读视图
轻量查询，不创建任务 worktree

L2

任务级 worktree
并发修改互不干扰，结束后合并

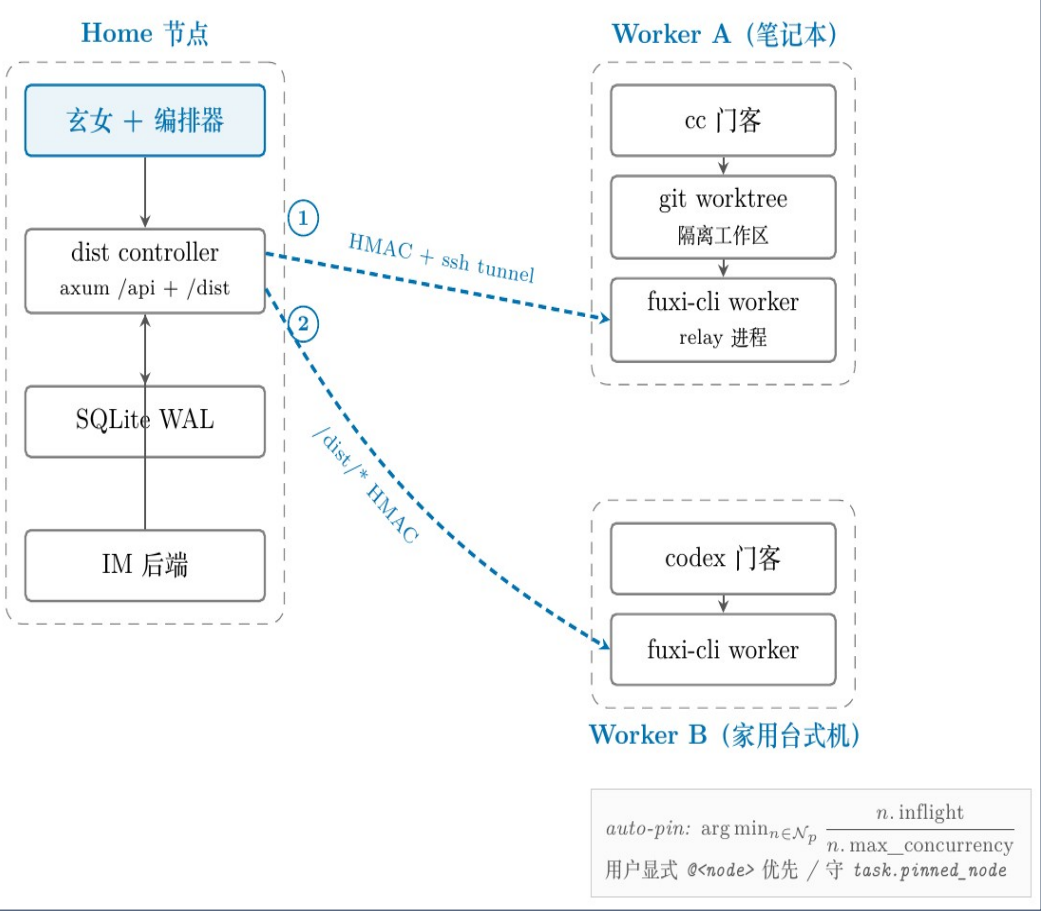
L3

角色级持久沙箱
复用构建缓存，适合长期角色

WS 反连让编排层能主动
send_message / cancel，
打破传统 stdio 只能等门客
poll 的限制。

跨节点扩展：单机优先，扩展点钩子化

默认单节点完整运行；多节点能力通过 trait 注入，不污染核心编排层。



关键机制

- Project.host_nodes 标记可执行节点。
- auto_pin_from_project 尊重用户显式 @node，自动逻辑不得覆盖显式意图。
- 节点选择按 inflight / max_concurrency 的饱和度最小化。
- HMAC-SHA256 + nonce cache + 常量时间比较，覆盖跨节点请求验签。

```
1: if |now - ts| > max_skew_ms then return ClockSkew // 同时防 replay 与 future-replay
2: expected <- HMAC_SHA256(K, method || path || ts || nonce || body)
3: if length(expected) != length(provided_sig) then return BadSignature
4: if not constant_time_eq(expected, provided_sig) then return BadSignature // 防 timing
oracle
5: if not NonceCache.check_and_insert(nonce) then return ReplayedNonce // 签名验过再查
nonce
```

当前边界：已跑通两节点 **sandbox**；**10+** 节点与不可靠链路故障转移是后续工作。

实验设计：先隔离 LLM，再测通讯层上限

合成基准测的是编排与通讯路径，不把模型推理时间混进系统指标。

模块	主要测试文件	测试目标
fuxi-a2a	roundtrip.rs	A2A wire 编解码与往返一致性
fuxi-events	integration.rs	事件发布、订阅与回放
fuxi-agent-cc	fixture_stream.rs、real_cc_smoke.rs	Claude Code 输出解析与真实启动
fuxi-agent-codex	fixture_stream.rs、real_codex_smoke.rs	Codex 输出解析与真实启动
fuxi-cli	chaos_resilience.rs 等 5 文件	CLI、网关重启、分布式与召贤 e2e
fuxi-im	router_smoke.rs 等 3 文件	IM API、WebSocket 与认证
fuxi-firehose	hub_roundtrip.rs	Firehose 事件 hub 往返
fuxi-memory	oracle_roundtrip.rs 等 4 文件	长期记忆、可复用模式与抽取
fuxi-orchestrator	dispatch.rs、deliverable_handoff.rs	任务派发与交付物交接
fuxi-scheduler	e2e_scheduler.rs	定时触发链路
fuxi-skills	loader.rs 等 4 文件	角色加载、暂存、登记与模板
fuxi-workspace	integration.rs	工作区创建与隔离

方法学

- 吞吐实验：每任务以 10 ms sleep 模拟，剥离 LLM 推理。
- 吞吐采用 5-run median；延迟采用 500 样本统计 p50 / p99。
- 覆盖正确性、实时性、吞吐扩展性、参数敏感性、事件总线压力。
- 真实 LLM Agent 的端到端吞吐主要受单次推理 1–30 s 主导。

worker_n	job_sleep	tasks_n	median_wall_ms	tasks/s	理论上限	Fuxi 损耗
1	10 ms	100	1227	81.50	100.00	18.5%
1	100 ms	50	5123	9.76	10.00	2.4%
1	1000 ms	10	10040	1.00	1.00	0.4%
4	10 ms	400	1212	330.03	400.00	17.5%
4	100 ms	200	5110	39.14	40.00	2.2%
4	1000 ms	40	10033	3.99	4.00	0.3%
8	10 ms	800	1203	665.00	800.00	16.9%
8	100 ms	400	5124	78.06	80.00	2.4%
8	1000 ms	80	10046	7.96	8.00	0.5%

cargo test

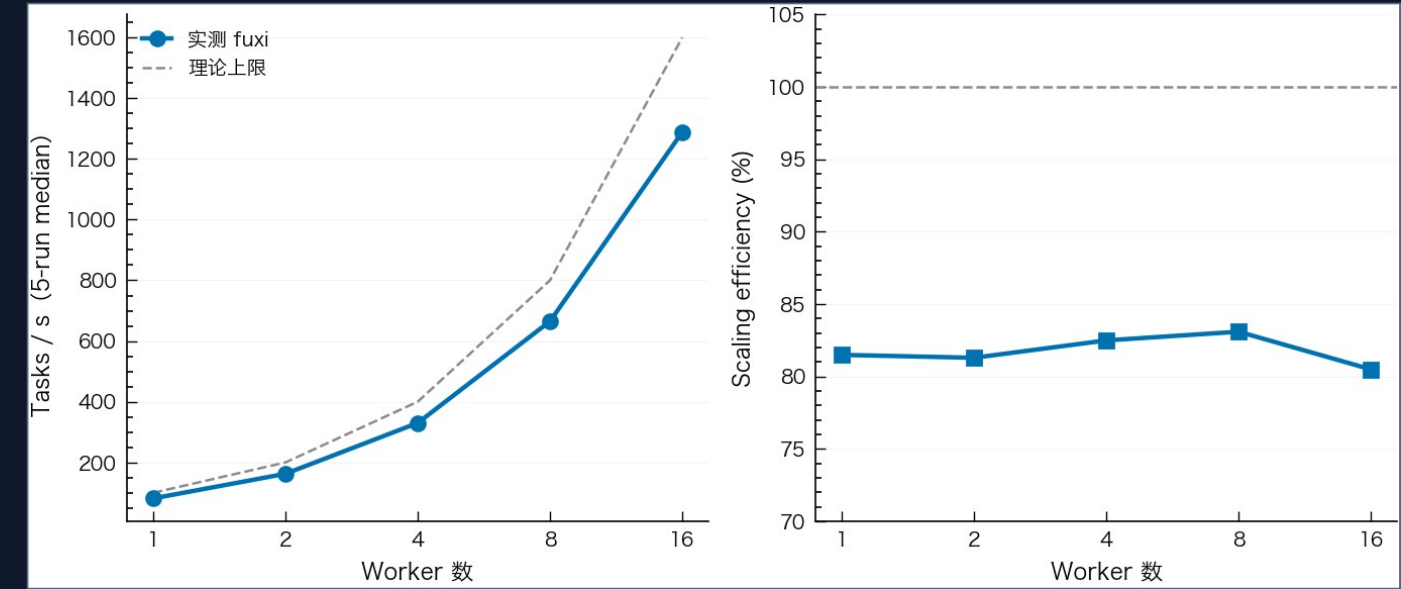
workspace all-targets 通过

500

延迟样本统计

结果一： Worker 扩展性没有在 16 worker 前饱和

8 worker 达 665 tasks/s；16 worker 达 1288.24 tasks/s。



665.00

8 worker × 10 ms 合成任务
调度路径吞吐 tasks/s

1288.24

16 worker 配置吞吐
tasks/s

80.5–83.1%

1→16 worker 扩展效率区间

未饱和

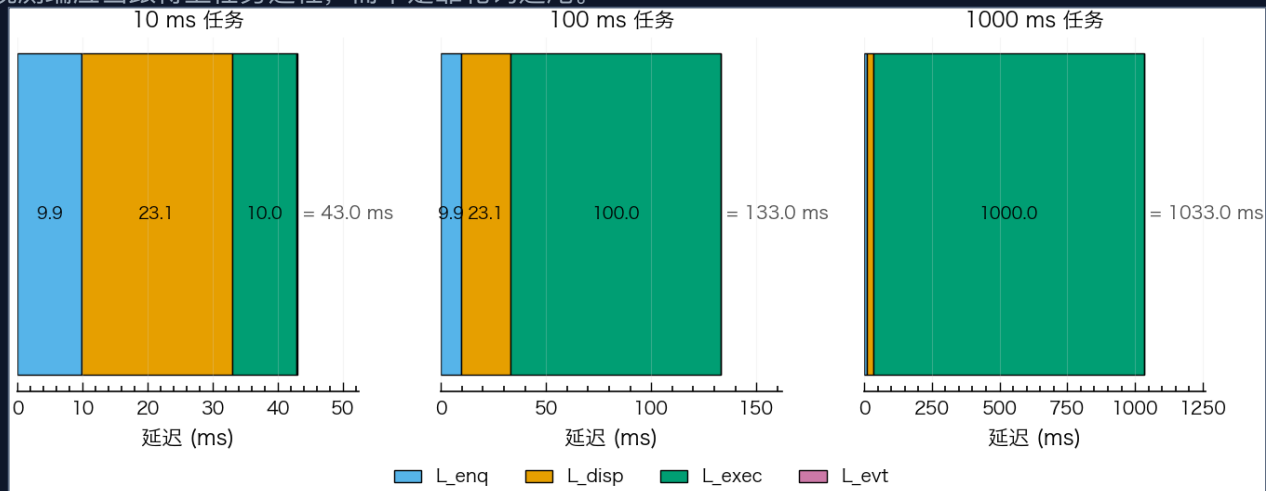
dispatcher 在该区间不
是主瓶颈

结论：通讯与调度路径在目标规模内有足够
余量，瓶颈主要会转移到真实 LLM 推理。

worker_n	tasks_n	median_wall_ms	tasks/s	理论上限	扩展效率
1	100	1227	81.50	100.00	81.5%
2	200	1230	162.60	200.00	81.3%
4	400	1212	330.03	400.00	82.5%
8	800	1203	665.00	800.00	83.1%
16	1600	1242	1288.24	1600.00	80.5%

结果二：事件流延迟比派发链路低三个数量级

观测端应当跟得上任务过程，而不是靠轮询追尾。



32.93 ms

任务派发 p50
含 HTTP / SQLite / 反序列化等

0.08 ms

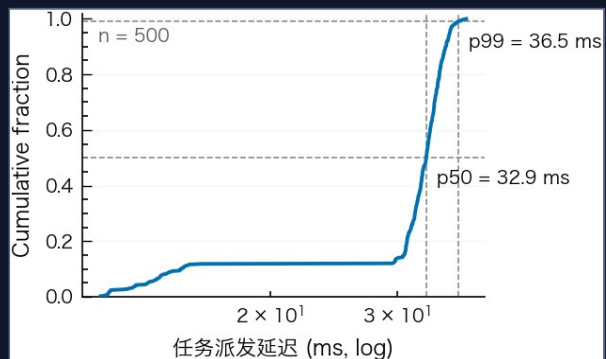
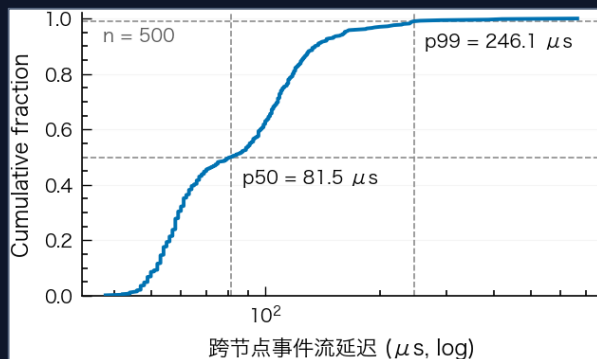
事件总线广播 p50

0.26 ms

事件总线广播 p99

410×

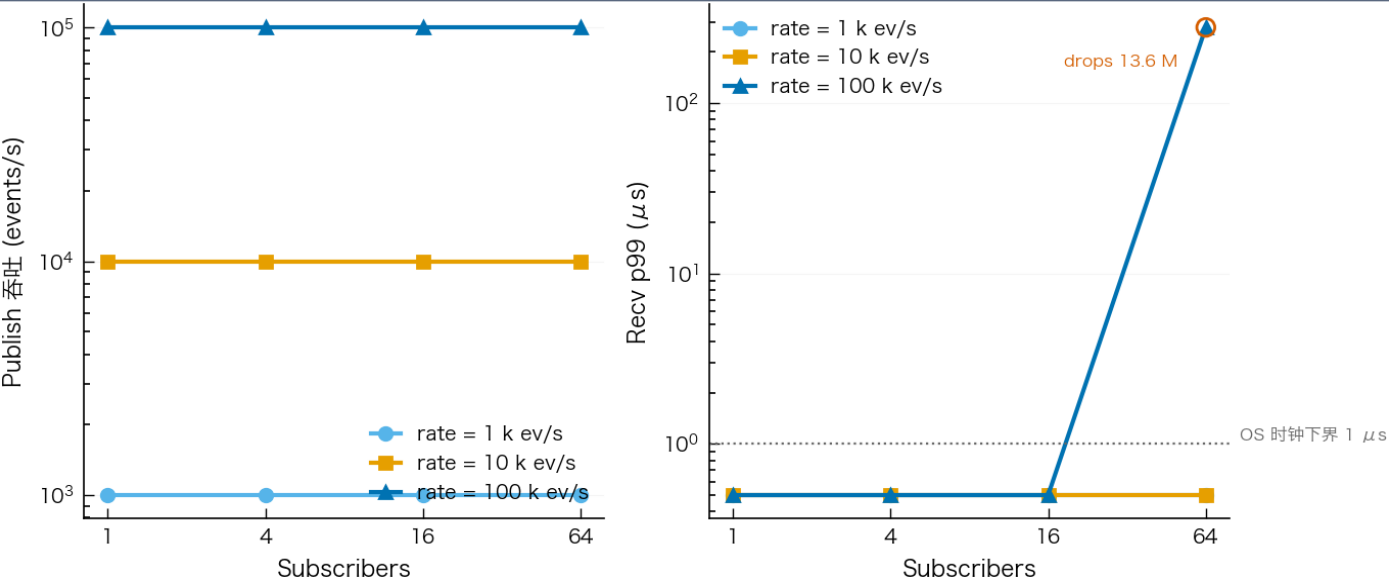
event_flow p50 与 dispatch
p50 差距



通讯层低延迟的意义：让 **Firehose**、**TUI**、**PWA** 能近实时显示过程事件。

结果三：事件总线压力测试给出容量边界

64 subscriber × 10k events/s 零丢帧； 16 subscriber × 100k events/s 可持续。



64 × 10k/s

持续负载下零丢帧

1.6M events/s

16 sub × 100k/s sustainable 边界

payload size 影响很小：small 256B 与 large 4096B 在零丢帧区间 p50/p99 差异不超过 0.1us，符合 Arc clone 扇出模型预期。

3 个数量级

高于实际工作负载估算余量

sub	n	rate(ev/s)	payload	publish tps	p50 us	p99 us	drops
1	1k	small	1000	< 1	< 1	0	
4	1k	small	1000	< 1	< 1	0	
16	1k	small	1000	< 1	< 1	0	
64	1k	small	1000	< 1	< 1	0	
16	100k	small	100000	< 1	< 1	0	
16	100k	large	100000	< 1	< 1	0	
64	10k	small	10000	< 1	< 1	0	
64	10k	large	10000	< 1	0.2	0	
64	100k	small	100000	163.5	278.4	13599792	
64	100k	large	100000	163.5	278.7	13598222	

局限与后续工作

答辩里主动承认边界，比被问住更好。

A2A 规范跟进

当前实现沿用早期 JSON-RPC binding；后续补齐 v1.0 PascalCase 与 one-of 互通测试。

安全模型

已有 worktree sandbox 与跨节点 HMAC；工具授权、数据访问控制、提示注入仍需统一策略。

分布式一致性

两节点 sandbox 已跑通；更大节点数、不可靠链路与故障转移尚未充分验证。

Agent 适配生态

已接 Claude Code / Codex；后续抽象更通用的 Agent CLI 适配框架。

路线没有因答辩结束而停止：论文记录的是 **Fuxi v1/v2** 阶段的工程化证据。

Q&A

谢谢各位老师

Fuxi 的差异化不在于“最快的 **Agent** 框架”，而在于把通讯底座从应用流程中显式剥离出来：事件流可观测可回放，协议层可独立部署，执行层可隔离扩展。

